📘Mini Proyecto: Refactorización, SOLID y Testing

🧠 ¿Por qué un Mini Proyecto?

Durante las últimas semanas estudiamos los conceptos por separado:

```
Testing
   ↓
Pruebas unitarias
   ↓
 Jest
   ↓
Refactorización
   ↓
  SRP
   ↓
  OCP
   ↓
  LSP
   ↓
  ISP
   ↓
  DIP
```

Pero en un proyecto real nadie nos dice: "En esta línea hay una violación de SRP."

Ni tampoco: "Aplica OCP en esta clase."

Nos encontramos simplemente con el código.

Nuestro trabajo consiste en analizarlo y tomar decisiones.

Por eso el objetivo de este proyecto no será: "Usar los cinco principios SOLID obligatoriamente."

Será: **Detectar problemas reales y justificar qué cambios conviene realizar.**

⚠️ SOLID no es una lista de requisitos

Esto es importante antes de empezar.

No vamos a evaluar de esta manera:

```
✓ Usó SRP
✓ Usó OCP
✓ Usó LSP
✓ Usó ISP
✓ Usó DIP
```

Porque eso puede llevar a modificar código que estaba perfectamente bien solamente para poder decir: "Acá usé SOLID."

Los principios son herramientas para analizar decisiones de diseño.

Por ejemplo, si una clase ya posee una responsabilidad clara, **no tenemos que dividirla solamente para demostrar SRP**.

🏪 Proyecto: Sistema de gestión de pedidos

Vamos a trabajar sobre un pequeño sistema para una tienda.

El sistema permite:

- registrar productos;
- crear pedidos;
- calcular el total;
- aplicar descuentos;
- seleccionar un tipo de envío;
- guardar pedidos;
- notificar al cliente.

El problema es que el sistema fue desarrollado rápidamente y, aunque funciona, su diseño comenzó a complicarse.

Nuestro trabajo será **mejorarlo**.

📦 Modelo inicial

🔍 Primera etapa – No tocar el código

Antes de programar, los grupos deberán analizarlo.

No pueden modificar ninguna línea durante los primeros minutos de la actividad.

Primero tienen que entenderlo.

🧠 Lo importante: los principios trabajan juntos

El sistema final no debería verse como:

Esta clase usa SRP.

Esta usa OCP.

Esta usa LSP.

En realidad, una decisión puede mejorar varios aspectos simultáneamente.

Por ejemplo:

```
Separar PedidoRepository
        ↓
Responsabilidad más clara
        ↓
       SRP

Inyectarlo en PedidoService
        ↓
Menor dependencia concreta
        ↓
       DIP
```

```
Agregar ApiPedidoRepository
sin modificar PedidoService
        ↓
       OCP
```

SOLID no son cinco compartimentos aislados.

📁 Estructura posible del proyecto

No la daría como obligación desde el comienzo, pero al finalizar podrían tener algo parecido a:

```
src/
│
├── models/
│   ├── Pedido.js
│   └── Producto.js
│
├── descuentos/
│   ├── DescuentoNormal.js
│   ├── DescuentoPremium.js
│   ├── DescuentoVIP.js
│   └── DescuentoMayorista.js
│
├── envios/
│   ├── EnvioNormal.js
│   ├── EnvioExpress.js
│   └── RetiroLocal.js
│
├── repositories/
│   └── LocalStoragePedidoRepository.js
│
├── notifications/
│   └── EmailNotificador.js
│
├── services/
│   └── PedidoService.js
│
└── main.js
```

```
tests/
│
├── descuentos.test.js
├── envios.test.js
└── pedidoService.test.js
```

Pero aceptaría otras estructuras si están bien justificadas.

Eso también es importante: **no convertir arquitectura en una receta mecánica**.

Modalidad del Mini Proyecto

Grupos de 2 o 3.

Cada grupo deberá crear un repositorio y trabajar mediante commits individuales.

No quiero:

```
1 commit: "proyecto terminado"
```

Quiero poder observar el proceso.

Por ejemplo:

```
test: agregar pruebas de descuentos

refactor: separar cálculo de descuentos

feat: agregar descuento mayorista

refactor: separar estrategias de envío

test: agregar pruebas de PedidoService
```

📌 Requisitos mínimos

El proyecto deberá contener:

1. Código inicial funcional.
2. Tests del comportamiento principal antes o durante la refactorización.
3. Separación de responsabilidades justificable.

4. Al menos un comportamiento extensible mediante polimorfismo.
5. Al menos una dependencia recibida desde afuera.
6. Incorporación de los dos nuevos requisitos:
    - Cliente Mayorista.
    - Retiro en Local.
7. Tests posteriores a la refactorización.
8. Uso de módulos `import/export`.
9. Repositorio GitHub.
10. Commits individuales de los integrantes.

📄 README

No necesitan escribir un informe enorme.

Debe explicar:

Integrantes

Descripción del sistema

Problemas encontrados

Refactorizaciones realizadas

Principios SOLID aplicados

Cómo ejecutar el proyecto

Cómo ejecutar los tests

Y especialmente: **¿Por qué realizaron esos cambios?**

🧠 No alcanza con nombrar el principio

Una respuesta cómo:"Aplicamos SRP."

no sería suficiente.

Deberían explicar: "La clase PedidoService calculaba descuentos, persistía pedidos y notificaba usuarios. Separamos la persistencia porque es una responsabilidad diferente y puede cambiar independientemente."

Eso demuestra comprensión.

🗣️ Cierre oral

10-15 minutos para que cada grupo explique muy brevemente una decisión.

No una defensa formal como el TP Integrador.

Preguntas como:

- ¿Cuál era el principal problema de `PedidoService`?
- ¿Qué refactorizaron primero y por qué?
- ¿Dónde aplicaron OCP?
- ¿Qué ventaja tuvo inyectar el repository?
- ¿Qué test les permitió comprobar que no habían roto el comportamiento?